Kirin

Hardware Integration Guide

Connecting the Raspberry Pi to the Physical Chess Board

Strydr Silverberg & Colin Dake

Colorado School of Mines
2026 ProtoFund

April 5, 2026

A step-by-step guide to wiring, configuring, and testing the Kirin autonomous chess system on a Raspberry Pi.

Contents

1 Overview **4**
- 1.1 What This Document Covers . . . 4
- 1.2 System Architecture . . . 4
- 1.3 Prerequisites . . . 5

2 Building the Software **6**
- 2.1 Clone and Build . . . 6
- 2.2 Install Dependencies . . . 6
- 2.3 Run the Tests . . . 6

3 Wiring the Hall Effect Sensors **8**
- 3.1 Sensor Overview . . . 8
- 3.2 Multiplexer Wiring . . . 8
  - 3.2.1 Shared Select Lines (Pi → All 6 Muxes) . . . 8
  - 3.2.2 Signal Output Lines (Each Mux → Pi) . . . 8
  - 3.2.3 Power and Enable . . . 9
- 3.3 Sensor-to-Channel Mapping . . . 9
- 3.4 A3144 Sensor Circuit . . . 9

4 Connecting the Gantry **11**
- 4.1 GRBL Connection . . . 11
- 4.2 GRBL Settings . . . 11
- 4.3 Electromagnet Control . . . 12
- 4.4 Physical Coordinate System . . . 12

5 First Power-On Procedure **13**
- 5.1 Step 1: Verify GPIO Chip . . . 13
- 5.2 Step 2: Start Kirin in Physical Mode . . . 13
- 5.3 Step 3: Run Sensor Diagnostics . . . 13
- 5.4 Step 4: Quick Scan . . . 14
- 5.5 Step 5: Test the Gantry . . . 14
- 5.6 Step 6: Gantry Coordinate Calibration . . . 14

6 Running Your First Game **15**
- 6.1 Stage 1: Manual Move Entry (No Sensors) . . . 15
- 6.2 Stage 2: Sensor-Based Play . . . 15
  - 6.2.1 What to Test . . . 15
- 6.3 Stage 3: Full Game . . . 16

7 Troubleshooting **17**
- 7.1 Sensor Issues . . . 17
- 7.2 Gantry Issues . . . 18
- 7.3 Move Detection Issues . . . 18

8 GPIO Pin Reference **19**

9 Software Command Reference **20**

10 Tunable Parameters **21**

Chapter 1

Overview

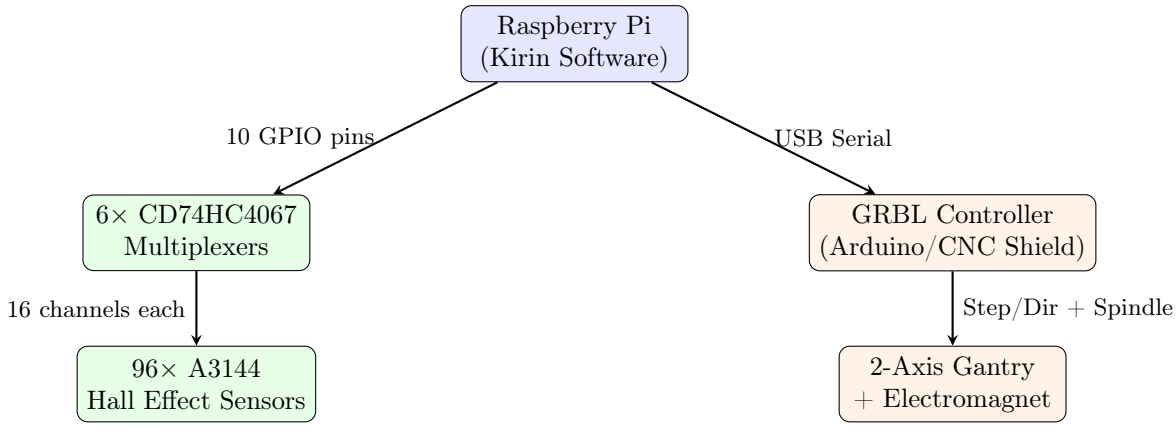
1.1 What This Document Covers

This guide walks through the complete process of connecting the Kirin software to the physical chess board hardware. By the end, you will have a Raspberry Pi running the Kirin engine, reading 96 hall effect sensors through multiplexers, and controlling a gantry system via GRBL over serial.

The system has three hardware subsystems that connect to the Pi:

1. **Hall Effect Sensors (96 total)** — detect piece presence on the board and in the two storage zones. Read through 6 multiplexers using 10 GPIO pins.
2. **GRBL Gantry Controller** — moves pieces on the physical board via an electromagnet on a 2-axis gantry. Communicates over USB serial.
3. **Electromagnet** — controlled by the GRBL board via the spindle output (M3/M5 commands).

1.2 System Architecture



1.3 Prerequisites

Item	**Details**
Raspberry Pi	Pi 4 or Pi 5 (any RAM)
OS	Raspberry Pi OS (64-bit recommended)
Compiler	GCC with C++17 support
CMake	Version 3.10 or later
libgpiod	`sudo apt install libgpiod-dev`
Serial port	USB connection to GRBL controller

Table 1.1: Required components

Chapter 2

Building the Software

2.1 Clone and Build

```
# Clone the repository
git clone https://github.com/your-repo/kirin.git
cd kirin

# Create build directory
mkdir build && cd build

# Configure with GPIO support
cmake .. -DCMAKE_BUILD_TYPE=Release \
         -DCMAKE_CXX_FLAGS="-DHAS_GPIOD"

# Build
make -j$(nproc)
```

Warning: The `-DHAS_GPIOD` flag is essential. Without it, the board scanner compiles in stub mode and all GPIO calls are no-ops. The build will succeed either way, but the sensors will not work without this flag.

2.2 Install Dependencies

```
# Install libgpiod for GPIO access (no root required at runtime)
sudo apt install libgpiod-dev

# Verify installation
dpkg -l | grep libgpiod
```

2.3 Run the Tests

Before connecting any hardware, verify the software is correct:

```
cd build

# Run all tests
ctest --output-on-failure

# Or run individually
./board_scanner_test      # Sensor move detection logic (62 tests)
./game_controller_test    # Engine-to-physical bridge
./board_interpreter_test  # Path planning
./engine_test             # Engine validation
./captured_piece_test     # Capture encoding
```

All tests should pass. The board scanner tests run entirely in simulation mode and do not require GPIO hardware.

Chapter 3

Wiring the Hall Effect Sensors

3.1 Sensor Overview

The system uses 96 A3144 hall effect sensors to detect the presence of magnets embedded in the chess pieces. The sensors are digital (output LOW when a magnet is detected, HIGH when no magnet is present) and require a 10K pull-up resistor each.

The 96 sensors are organized as follows:

Mux	Coverage	Channels	GPIO (SIG)
0	Board ranks 8–7 (a8–h7)	0–15	GPIO 5
1	Board ranks 6–5 (a6–h5)	0–15	GPIO 6
2	Board ranks 4–3 (a4–h3)	0–15	GPIO 13
3	Board ranks 2–1 (a2–h1)	0–15	GPIO 19
4	Black storage zone (16 slots)	0–15	GPIO 26
5	White storage zone (16 slots)	0–15	GPIO 16

Table 3.1: Multiplexer assignments

3.2 Multiplexer Wiring

All six CD74HC4067 multiplexers share four address/select lines. The Pi sets these four lines (S0–S3) to select a channel (0–15), then reads the SIG output from each mux simultaneously.

3.2.1 Shared Select Lines (Pi → All 6 Muxes)

Select Line	Pi GPIO (BCM)	Mux Pin
S0	GPIO 17	Pin 10 (S0) on all muxes
S1	GPIO 27	Pin 11 (S1) on all muxes
S2	GPIO 22	Pin 14 (S2) on all muxes
S3	GPIO 23	Pin 15 (S3) on all muxes

Table 3.2: Select line wiring (active on all 6 muxes)

3.2.2 Signal Output Lines (Each Mux → Pi)

Each mux has a single SIG output that goes to its own GPIO pin on the Pi:

Mux	SIG → Pi GPIO	Direction
0	GPIO 5	Input (Pi reads)
1	GPIO 6	Input
2	GPIO 13	Input
3	GPIO 19	Input
4	GPIO 26	Input
5	GPIO 16	Input

Table 3.3: Signal output wiring

3.2.3 Power and Enable

- Connect VCC on each mux to 3.3V (the Pi's GPIO operates at 3.3V).
- Connect GND on each mux to the Pi's GND.
- Connect the EN (enable) pin on each mux to GND (active-low enable, so grounding it keeps the mux always enabled).

Warning: Do not power the muxes from 5V. The CD74HC4067 will work at 3.3V, and connecting a 5V signal to the Pi's GPIO pins can damage the Pi.

3.3 Sensor-to-Channel Mapping

Within each board mux, the 16 channels map to squares as follows:

Channel	Square (Mux 0)	Channel	Square (Mux 0)
0	a8	8	a7
1	b8	9	b7
2	c8	10	c7
3	d8	11	d7
4	e8	12	e7
5	f8	13	f7
6	g8	14	g7
7	h8	15	h7

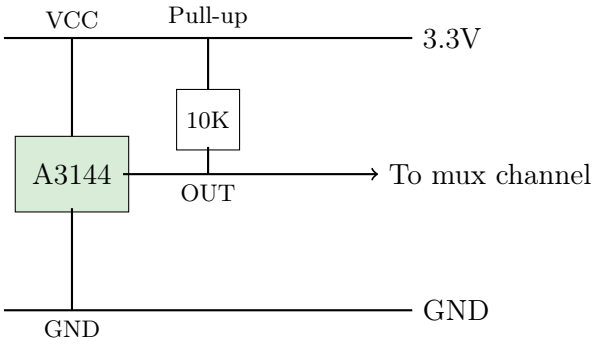
Table 3.4: Channel-to-square mapping (Mux 0 shown; pattern repeats for Mux 1–3)

The pattern is: channels 0–7 map to the higher rank (file a through h), channels 8–15 map to the lower rank. For storage muxes (4 and 5), channels 0–7 are back-rank piece slots and channels 8–15 are pawn slots.

Tip: If your physical wiring doesn't match this channel order, you have two options: re-wire the sensors, or modify `toSquareIndex()` in `board_scanner.cpp` to match your actual wiring. Re-wiring is usually easier.

3.4 A3144 Sensor Circuit

Each A3144 sensor has three pins: VCC, GND, and OUTPUT. The output is open-collector, so it needs a pull-up resistor:



When a magnet is present, the A3144 pulls its output LOW. When no magnet is present, the pull-up resistor holds the output HIGH. The software inverts this logic: LOW = piece present, HIGH = empty.

Chapter 4

Connecting the Gantry

4.1 GRBL Connection

The gantry controller (typically an Arduino Uno with a CNC shield running GRBL firmware) connects to the Pi via USB. This appears as a serial port, usually `/dev/ttyUSB0` or `/dev/ttyACM0`.

```
# Find the serial port
ls /dev/ttyUSB* /dev/ttyACM*

# Check permissions (your user must be in the 'dialout' group)
groups $USER | grep dialout

# If not in the group:
sudo usermod -aG dialout $USER
# Log out and back in for this to take effect
```

4.2 GRBL Settings

The Kirin software communicates at 115200 baud (GRBL default). Verify the GRBL settings match the physical gantry dimensions. Connect to GRBL with a serial terminal to check:

```
# Connect with minicom (or screen, picocom, etc.)
minicom -D /dev/ttyUSB0 -b 115200

# In the GRBL console, type:
$$
# This prints all settings
```

Key settings to verify:

Setting	Parameter	Notes
$100	X steps/inch	Must match your stepper + lead screw
$101	Y steps/inch	Must match your stepper + lead screw
$110	X max rate (in/min)	Software uses 1000 in/min default
$111	Y max rate (in/min)	Must be $\geq$ 1000
$120	X acceleration (in/s^2)	Start with 10, increase after testing
$121	Y acceleration (in/s^2)	Start with 10, increase after testing
$130	X max travel (in)	Should be $\geq$ 22 (rail travel)
$131	Y max travel (in)	Should be $\geq$ 22
$22	Homing cycle	Set to 1 (enabled)

Table 4.1: Critical GRBL settings

4.3 Electromagnet Control

The electromagnet is controlled through GRBL's spindle output:

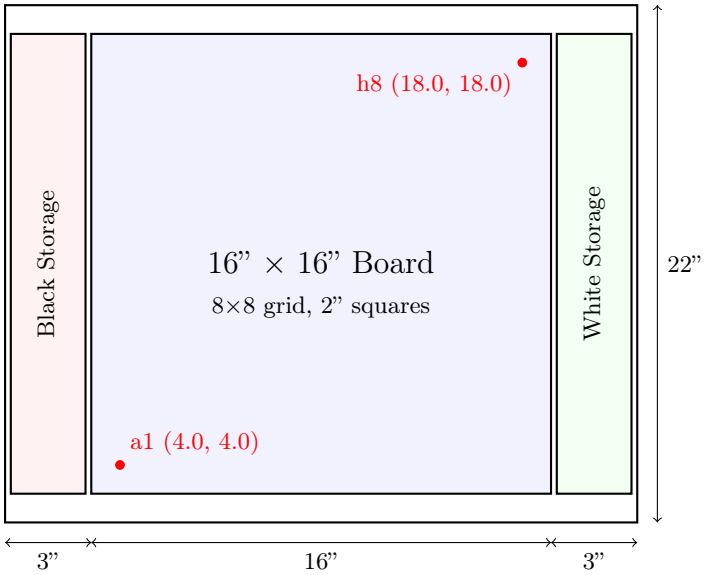
- `M3` = Magnet ON (spindle CW — repurposed for electromagnet)
- `M5` = Magnet OFF (spindle stop)

The CNC shield's spindle enable pin drives a MOSFET or relay that controls the electromagnet. Verify this works before running any game:

```
# In the GRBL console:
M3      # Magnet should engage
G4 P2000  # Wait 2 seconds
M5      # Magnet should release
```

4.4 Physical Coordinate System

The software maps board squares to physical coordinates in inches. The board layout is:



The coordinate system origin (0,0) is at the bottom-left of the rail travel. Square a1 is centered at (4.0", 4.0") and each square is 2" wide. Verify that the gantry homes to (0,0) and that commanding `G1 X4.0 Y4.0` positions the magnet over the center of a1.

Chapter 5

First Power-On Procedure

5.1 Step 1: Verify GPIO Chip

The Pi 5 and Pi 4 use different GPIO chip paths. The software auto-detects, but verify:

```
# Pi 5:
ls /dev/gpiochip4  # Should exist

# Pi 4 and earlier:
ls /dev/gpiochip0  # Should exist

# List all GPIO chips:
gpiodetect
```

5.2 Step 2: Start Kirin in Physical Mode

```
cd build
./kirin --physical /dev/ttyUSB0
```

You should see:

```
Kirin Autonomous Chess System v0.3
Physical Board Mode

Connecting to gantry on /dev/ttyUSB0...
Connected to gantry on /dev/ttyUSB0
Initializing board scanner...
[SCANNER] GPIO initialized successfully (6 muxes: 4 board + 2 storage)
  Select lines: GPIO 17, 27, 22, 23
  Board muxes:  GPIO 5, 6, 13, 19
  Storage muxes: GPIO 26 (black), 16 (white)
Board scanner initialized
Homing gantry...
Homing complete

Ready. Type 'help' for commands.
```

If the scanner fails to initialize, check your wiring and ensure `-DHAS_GPIOD` was used during compilation.

5.3 Step 3: Run Sensor Diagnostics

```
kirin> diag
```

This reads every sensor individually and prints the result. Walk through each square with a magnet (or chess piece) and verify:

- Each channel maps to the correct square
- "PIECE" appears when the magnet is present, "empty" when removed
- No stuck sensors (always reading PIECE or always empty)
- Storage zone sensors respond correctly

5.4 Step 4: Quick Scan

Place all 32 pieces on the board in the starting position, then:

```
kirin> scan
```

You should see:

```
  Sensor Scan:
    a b c d e f g h
  8 # # # # # # # # 8
  7 # # # # # # # # 7
  6 . . . . . . . . 6
  5 . . . . . . . . 5
  4 . . . . . . . . 4
  3 . . . . . . . . 3
  2 # # # # # # # # 2
  1 # # # # # # # # 1
    a b c d e f g h
  Pieces detected: 32
```

If any squares are wrong, re-check the wiring for that mux/channel.

5.5 Step 5: Test the Gantry

```
kirin> test
```

This moves the gantry to e4, then d5, engages and disengages the magnet, and returns home. Watch for:

- Smooth motion with no skipped steps
- Correct positioning over the square centers
- Magnet picks up and releases cleanly
- No collisions with pieces or board edges

5.6 Step 6: Gantry Coordinate Calibration

If the gantry doesn't align with the square centers, you need to adjust either the GRBL steps/inch settings or the software constants. Place a piece on a1 and command:

```
kirin> home
```

Then type `test`. The gantry should move to e4 (center of the board). If it's offset, adjust `A1_CENTER_X` and `A1_CENTER_Y` in `gantry_controller.h`, or recalibrate your GRBL $100/$101 steps/inch settings.

> **Tip:** Get a1 and h8 aligned first. Since the coordinate system is linear, if these two corners are correct, all 64 squares will be correct.

Chapter 6

Running Your First Game

6.1 Stage 1: Manual Move Entry (No Sensors)

Start with manual move input to verify the gantry executes moves correctly:

```
kirin> newgame black
```

This sets up the board (gantry moves all pieces from storage to starting squares), then the engine plays black. You type your moves:

```
kirin> move e2e4
kirin> move d2d4
kirin> move g1f3
```

After each of your moves, the engine will think and physically move a piece in response. Watch for correct piece movement, especially during captures (the captured piece should go to the correct storage slot).

6.2 Stage 2: Sensor-Based Play

Once manual mode works, switch to the full sensor experience:

```
kirin> play black
```

Now make your moves by physically moving pieces on the board. The system will detect your move automatically. The terminal will show:

```
Move 1. -- Your turn. Make your move on the board.
Waiting for your move...
Detected move: e2e4
```

6.2.1 What to Test

1. **Normal moves**: Pick up a pawn, place it on a new square.
2. **Pondering**: Pick up a piece, hold it for 10+ seconds, put it back. The system should *not* commit a move.
3. **Take-back**: Pick up one piece, put it back, pick up a different piece and move it. Only the final move should register.
4. **Captures**: Pick up the opponent's piece, place it in the storage zone, then move your piece to the vacated square.
5. **Castling**: Move the king two squares, then move the rook. The system waits until both pieces are in their final positions.
6. **Illegal state**: Move a piece to an illegal square. After 10 seconds, the system will alert you.

6.3 Stage 3: Full Game

Play a complete game to checkmate or stalemate. The system will announce the result:

```
==============================
  CHECKMATE -- White wins!
==============================
```

Chapter 7

Troubleshooting

7.1 Sensor Issues

Symptom	Solution
All sensors read empty	Check mux power (3.3V), verify EN pin is grounded, check select line wiring
One mux reads all empty	Check that mux's SIG output wire to the Pi GPIO pin
Random noise / flickering	Increase `DEBOUNCE_SCANS` (default 3) or `DEBOUNCE_DELAY_MS` (default 50ms) in `board_scanner.h`
Sensors read wrong squares	Your channel-to-square wiring doesn't match the expected layout. Run `diag` and compare against the mapping table. Either re-wire or modify `toSquareIndex()`
Sensors trigger from gantry magnet	The gantry electromagnet is interfering with nearby hall sensors. Add shielding around the gantry magnet, increase debounce, or only scan when the gantry is idle
Scanner init fails	Verify `libgpiod-dev` is installed and the build used `-DHAS_GPIOD`. Check that the GPIO chip exists (`/dev/gpiochip4` on Pi 5, `/dev/gpiochip0` on Pi 4)

Table 7.1: Common sensor issues

7.2 Gantry Issues

Symptom	Solution
"Failed to connect to gantry"	Check USB cable, verify serial port path (`ls /dev/ttyUSB*`), ensure user is in `dialout` group
Homing fails	Verify limit switches are connected and GRBL setting $22=1 (homing enabled)
Gantry moves to wrong position	Recalibrate GRBL steps/inch ($100, $101) or adjust `A1_CENTER_X`/`A1_CENTER_Y` in the software
Magnet doesn't engage	Check GRBL spindle output wiring, test with `M3`/`M5` in a serial console
Pieces dragged/knocked over	Reduce `FEED_RATE` (default 1000 in/min), verify magnet strength is appropriate
GRBL alarm triggered	The software auto-resets on alarm. Check error description in terminal output. Common cause: soft limits exceeded (piece near board edge)

Table 7.2: Common gantry issues

7.3 Move Detection Issues

Symptom	Solution
System doesn't detect my move	Verify the sensor under the destination square works (`diag`). Check that you placed the captured piece in storage (required for capture detection).
System hangs waiting for move	If it's a rare ambiguous capture (queen can capture two different pieces from the same square), fall back to typed input: press Ctrl+C and use `move` command.
"Board state not recognized" alert	You've left the board in a state that doesn't correspond to any legal move for 10+ seconds. Fix piece placement.
Board mismatch after engine move	The gantry dropped a piece. The system will show which squares are wrong. Fix the pieces manually and type `continue`.

Table 7.3: Common move detection issues

Chapter 8

GPIO Pin Reference

Complete pin assignment table using BCM (Broadcom) numbering:

BCM Pin	Direction	Function	Connects To
17	Output	Select S0	S0 on all 6 muxes
27	Output	Select S1	S1 on all 6 muxes
22	Output	Select S2	S2 on all 6 muxes
23	Output	Select S3	S3 on all 6 muxes
5	Input	Mux 0 SIG	Board ranks 8–7
6	Input	Mux 1 SIG	Board ranks 6–5
13	Input	Mux 2 SIG	Board ranks 4–3
19	Input	Mux 3 SIG	Board ranks 2–1
26	Input	Mux 4 SIG	Black storage zone
16	Input	Mux 5 SIG	White storage zone

Table 8.1: All GPIO pin assignments (10 pins total)

Note: These pin assignments can be overridden by passing custom arrays to the `BoardScanner` constructor. The defaults are defined in `board_scanner.h`. If you need to change them, modify the `DEFAULT_PIN_*` constants and rebuild, or pass them at construction time.

Raspberry Pi Header Pin	BCM GPIO
Pin 11	GPIO 17 (S0)
Pin 13	GPIO 27 (S1)
Pin 15	GPIO 22 (S2)
Pin 16	GPIO 23 (S3)
Pin 29	GPIO 5 (Mux 0)
Pin 31	GPIO 6 (Mux 1)
Pin 33	GPIO 13 (Mux 2)
Pin 35	GPIO 19 (Mux 3)
Pin 37	GPIO 26 (Mux 4)
Pin 36	GPIO 16 (Mux 5)

Table 8.2: BCM to physical header pin mapping

Chapter 9

Software Command Reference

Quick reference for all commands available in physical mode:

Command	Description
`play [white|black]`	Start a sensor-based game. Engine plays the specified color (default: white). Moves are detected automatically via hall effect sensors.
`newgame [white|black]`	Start a game with manual (typed) move entry. Useful as a fallback if sensors aren't working.
`move <uci>`	Manually enter a move in UCI format (e.g., `move e2e4`, `move e7e8q` for promotion).
`go`	Engine makes one move.
`scan`	Scan and display all 96 sensors (board + storage zones).
`diag`	Run full sensor diagnostic — reads each sensor individually with channel-by-channel output.
`board`	Display the engine's internal board position.
`physical`	Display the physical board occupancy tracking.
`fen <fen>`	Set position from a FEN string.
`home`	Home the gantry to (0,0).
`test`	Run gantry hardware test (move to e4, d5, test magnet).
`quit`	Exit the program.

Table 9.1: Physical mode commands

Chapter 10

Tunable Parameters

These constants in `board_scanner.h` and `gantry_controller.h` may need adjustment based on your specific hardware:

Constant	Default	Purpose
`MUX_SETTLE_US`	10 μs	Time to wait after changing mux address before reading. Increase if readings are noisy.
`DEBOUNCE_SCANS`	3	Number of consecutive identical scans required before accepting a reading as stable.
`DEBOUNCE_DELAY_MS`	50 ms	Delay between debounce scans. Total debounce time = `SCANS` $\times$ `DELAY`.
`POLL_INTERVAL_MS`	20 ms	How often to poll sensors during `waitForLegalMove`.
`ILLEGAL_STATE_ALERT_MS`	10,000 ms	How long the board can sit in an unrecognized state before alerting the player.
`FEED_RATE`	1000 in/min	Gantry movement speed. Reduce if the gantry skips steps or knocks pieces over.
`SQUARE_SIZE`	2.0 in	Physical size of each board square.
`BOARD_MARGIN`	3.0 in	Distance from rail edge to first board square.

Table 10.1: Tunable hardware parameters